Day 92 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 14, Day 2 reference

Watch: Let's build GPT: from scratch, in code, spelled out (Andrej Karpathy) | Let's reproduce GPT-2 (124M) (Andrej Karpathy)

`day92_training_mini_gpt.py`, segment by segment, in the order `main()` uses things. Day 90's model is imported and explained there; this covers the schedule, the measurement and running jobs in parallel. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: T = context length · peak = the largest learning rate · warm-up = steps to reach it · floor = the final fraction of the peak

Segment 0 — Setup

Importing Day 90, and the two models

CODE

```
sys.path.insert(0, os.path.join(HERE, "..", "week-13-tokenization-embeddings"))
with contextlib.redirect_stdout(io.StringIO()):
    import day90_tiny_gpt as d90
SMALL = dict(T=64, B=32, d=64, h=4, ff=256, layers=2)
BIG = dict(T=128, B=16, d=128, h=4, ff=512, layers=4)
STEPS, CHECK_EVERY = 5000, 500
WARMUP, FLOOR = 200, 0.1
SWEEP, SWEEP_STEPS = (1e-3, 3e-3, 6e-3), 300
PEAK_SMALL = 3e-3
```

- `SMALL = dict(T=64, B=32, d=64, h=4, ff=256, layers=2)` — a model's settings as a dictionary, passed around as one object.
- `BIG = dict(T=128, B=16, ...)` — twice the context, half the batch: the same 2,048 tokens per step.

Segment 1 — The schedule and the measurement

lr_schedule: warm-up, then a cosine

MATH

$$lr(s) = peak \cdot (s + 1) / W \text{ for } s < W; \quad lr(s) = peak \cdot [f + (1 - f) \cdot (1 + \cos(\pi\,(s - W) / (S - W))) / 2] \text{ after}$$

CODE

```
def lr_schedule(step, peak, warmup=WARMUP, total=STEPS, floor=FLOOR):
    if step < warmup:
        return peak * (step + 1) / warmup
    progress = (step - warmup) / max(1, total - warmup)
    return peak * (floor + (1 - floor) * 0.5 * (1 + np.cos(np.pi * min(1.0, progress))))
```

- `peak * (step + 1) / warmup` — a straight line from almost 0 up to the peak.
- `0.5 * (1 + np.cos(np.pi * progress))` — goes from 1 to 0 as progress goes from 0 to 1: half a cosine.
- `min(1.0, progress)` — guards against steps past the end.

eval_bits: full context for every scored token

MATH

$$bits = -\Sigma \log p(x_t \mid x_{t-T/2..t-1} \text{ or more}) / (count \cdot \ln 2)$$

CODE

```
def eval_bits(P, ids, T, h, max_windows=None):
    half = T // 2
    starts = np.arange(0, len(ids) - T - 1, half)[:max_windows]
    nats, count = 0.0, 0
    for s in range(0, len(starts), 32):
        st = starts[s:s + 32]
        x = np.stack([ids[i:i + T] for i in st]); y = np.stack([ids[i + 1:i + T + 1] for i in st])
        logp = d90.log_softmax(d90.gpt_forward(x, P, h)[0][:, half:])
        nats -= np.take_along_axis(logp, y[:, half:, None], axis=-1).sum()
        count += y[:, half:].size
    return nats / count / np.log(2)
```

- `np.arange(0, len(ids) - T - 1, half)` — a window every T/2 tokens, so consecutive windows overlap by half.
- `[:, half:]` — only the second half of each window is scored: every scored token has at least T/2 tokens before it.
- `np.take_along_axis(logp, y[:, half:, None], axis=-1)` — the log-probability of the token that really came next.

Segment 2 — Training in parallel

train_run: one job, in its own process

CODE

```
def train_run(job):
    name, cfg, peak, schedule, steps, seed = job
    _, _, vocab, tr, va = data()
    tr_eval = tr[:len(va)]
    with threadpool_limits(1, "blas"):
        rng = np.random.RandomState(seed)
        P = d90.init_gpt(rng, len(vocab), d=cfg["d"], ff=cfg["ff"], layers=cfg["layers"],
            t=cfg["T"])
        state, recent, busy = {"t": 0, "m": {}, "v": {}}, [], 0.0
        measure = steps > SWEEP_STEPS
        curve = [(0, 0.0, np.nan, eval_bits(P, va, cfg["T"], cfg["h"]), eval_bits(P, tr_eval,
            cfg["T"], cfg["h"]))] if measure else []
        T, B = cfg["T"], cfg["B"]
        for step in range(1, steps + 1):
            t0 = time.perf_counter()
            lr = lr_schedule(step - 1, peak, warmup=min(WARMUP, steps // 3), total=steps) if
                schedule else peak
            o = rng.randint(0, len(tr) - T - 1, B)
            x = np.stack([tr[i:i + T] for i in o]); y = np.stack([tr[i + 1:i + T + 1] for i in o])
            loss, G = d90.gpt_loss_grads(x, y, P, h=cfg["h"])
            d90.adam_step(P, G, state, lr); recent.append(loss)
            busy += time.perf_counter() - t0
            if (measure and step % CHECK_EVERY == 0) or (not measure and step % 50 == 0):
                row = (step, lr, np.mean(recent) / np.log(2))
                if measure:
                    row += (eval_bits(P, va, T, cfg["h"]), eval_bits(P, tr_eval, T, cfg["h"]))
                curve.append(row); recent = []
    return dict(name=name, curve=np.array(curve), P=P, sec=busy / steps, weights=sum(a.size for _,
        a in d90.w12.leaves(P)))
```

- `name, cfg, peak, schedule, steps, seed = job` — unpacking a tuple: one job description, six values.

- `tr_eval = tr[:len(va)]` — a slice of TRAINING text as long as the held-out text, measured the same way: the overfitting check.
- `lr_schedule(...) if schedule else peak` — a conditional expression picks the scheduled or the constant rate.
- `return dict(...)` — everything the parent process needs travels back as one dictionary.

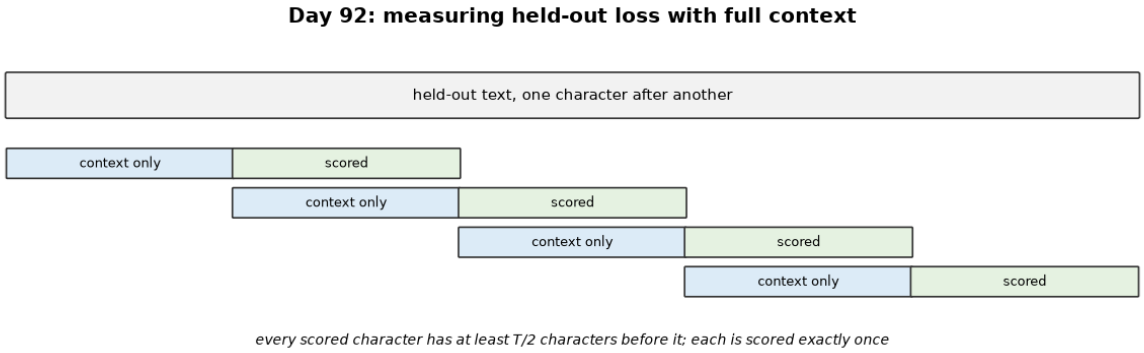
run_parallel: several jobs at once

CODE

```
def run_parallel(jobs):
    with mp.get_context("spawn").Pool(len(jobs)) as pool:
        return pool.map(train_run, jobs)
```

- `mp.get_context("spawn").Pool(len(jobs))` — one fresh Python process per job; 'spawn' works the same on Windows, macOS and Linux.
- `pool.map(train_run, jobs)` — runs train_run on every job at the same time and returns the results in the jobs' order.
- `if __name__ == "__main__":` — essential with spawn: each worker imports the script, and must not start main() itself.

SYSTEM DIAGRAM



Held-out loss with full context: windows every T/2, only their second halves scored.

Segment 3 — Reporting and saving

demo_honest: best checkpoint, gap, perplexity with its unit

MATH

$$ppl_{char} = 2^{b}; \quad ppl_{word} = 2^{b \cdot (\text{characters per word})}$$

CODE

```
cpw = len(tr_text) / len(words)
print(f"Perplexity = 2^(bits per token): the number of equally likely choices the model is, on
    average, choosing "
      f"between. Per character it is small; per word it is large. The training text has {cpw:.2f}
          characters per "
      "word (spaces and newlines included), so bits per word = bits per character x that.")
print(f"\n{'run':<20}{'final held-out':>15}{'best (step)':>15}{'train
    text':>12}{'gap':>7}{'ppl/char':>10}{'ppl/word':>10}")
for r in res:
    c = r["curve"]; fin = c[-1]; b = int(np.argmin(c[:, 3]))
    print(f"{r['name']:<20}{fin[3]:>15.3f}{f'{c[b, 3]:.3f} ({int(c[b,
        0])})':>15}{fin[4]:>12.3f}{fin[3] - fin[4]:>7.3f}"
          f"{2 ** fin[3]:>10.2f}{2 ** (fin[3] * cpw):>10.0f}")
```

- `np.argmin(c[:, 3])` — the check with the lowest held-out loss: the best checkpoint, which may not be the last.
- `2 ** fin[3]` — perplexity per character: 2 to the bits per character.
- `2 ** (fin[3] * cpw)` — perplexity per word: the same model's bits per word, characters per word × bits per character.

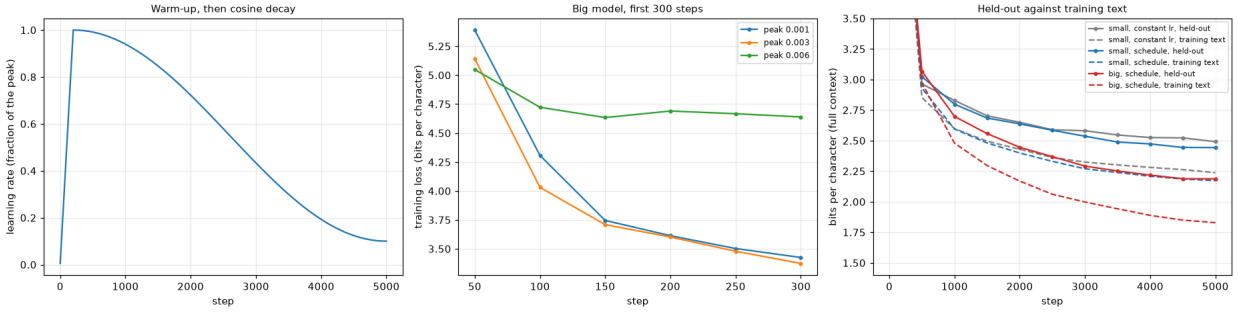
save_model: weights to a file

CODE

```
def save_model(r, vocab):
    flat = {k.replace(".", "__"): v.astype(np.float32) for k, v in d90.w12.leaves(r["P"])}
    np.savez_compressed(os.path.join(HERE, "day92_mini_gpt.npz"), vocab=np.array(vocab), **flat,
        **{f"cfg__{k}": v for k, v in BIG.items()})
    print(f"\nsaved the big model to day92_mini_gpt.npz ({r['weights']} weights as float32, its
        vocabulary and settings)")
```

- `k.replace(".", "__")` — 'blocks.0.Wq' becomes 'blocks__0__Wq': a flat name the file format accepts.
- `v.astype(np.float32)` — half the size; float32 keeps about 7 significant digits, plenty for reuse.
- `np.savez_compressed(path, **flat)` — ** spreads a dictionary into keyword arguments: one named array each.

REAL GRAPH



Real graph from the script: the schedule, the sweep, and held-out against training text.

Quick Syntax Glossary — new or reinforced constructs

- `multiprocessing Pool.map(f, items)` — run f on every item in separate processes, results in order.
- `if __name__ == "__main__":` — code that runs only when the file is run, not when it is imported.
- `np.cos(np.pi * x)` — the cosine, used for the decay.
- `np.argmin(a)` — the position of the smallest value.
- `np.savez_compressed(path, **arrays)` — many named arrays in one compressed file.
- `f(**d)` — a dictionary's items passed as keyword arguments.

End of Day 92 syntax reference. For the experiments and what they showed, see the Day 92 complete guide.